OCC: SOFTWARE ENGINEER YEAR 2

SAQA ID

NQF 6

Software design and testing C#

SDT621

SUMMATIVE ASSESSMENT

2026

CTU training solutions | Private Tertiary Education

Table of Contents

DECLARATION OF AUTHENTICITY

I (FULL NAME) Onthatile Tshegofatso Mashego

hereby declare that the contents of this assessment
are entirely my own work, completed by me without any paraphrasing/copying, or presented as my own work not accessed from any **AI Apps, example ChatGPT, Co-pilot, Perplexity, or any other App.**

I have read and understood the assessment requirements and the Examination Rules and Regulations.

Signature: Onthatile T. Mashego Onthatile T. Mashego

Date: 5/27/2026

QUALIFICATION AND ASSESSMENT DETAILS

Programme Title	OCC: SOFTWARE ENGINEER
Module Title	Software design and testing C#
Module Code	SDT621
NQF Level	6
Assessment Type	SUMMATIVE ASSESSMENT
Hand Out Date	27 / 0 5 / 2026
Hand in Date	27 / 05 / 2026
Total Marks	100
Assessment Lead Developer	Lusukama Selemani
Internal Moderator	Faith Muwishi

Student Name and Surname	Onthatile Mashego
Student Number	20240103

Instructions:

Test or Debug Source Code to Ensure Client Needs Are Met

Please read the following instructions carefully before attempting this assessment:

- Read each question carefully and consider the mark allocation before answering.
- Answer all questions in the assessment.
- Provide clear, well-structured C# code and explanations where required.
- Apply your understanding of software testing and debugging techniques when solving the given scenarios.
- Ensure that your solutions meet the client requirements described in the scenario.
- Use meaningful variable names, proper formatting, and correct programming logic.
- Test your application before submission to confirm that it runs correctly.

Submission Requirements

- For your final submission, you must include:
- A signed Declaration of Authenticity (if not already included in the assessment document provided)
- Your completed Answer Sheet (PDF document) (preferably the official template provided with this assessment)
- Your Visual Studio project files (compressed as a ZIP file)

Evidence Collection Requirements

To support your assessment submission, you must:

- Take a screenshot of the program output for each question
- Ensure each screenshot shows the full screen, including the system date and time
- Ensure screenshots clearly demonstrate correct program execution
- Do NOT submit screenshots of your source code
- Submit your complete Visual Studio project folder as a ZIP file

GitHub Submission Requirement

You are required to:

- Push your project code to GitHub
- Ensure the repository contains the correct and complete solution
- Share the GitHub repository link in your submission for verification and review
- Example: https://github.com/your-username/project-name

Summative Assessment Criteria:

Applied Knowledge

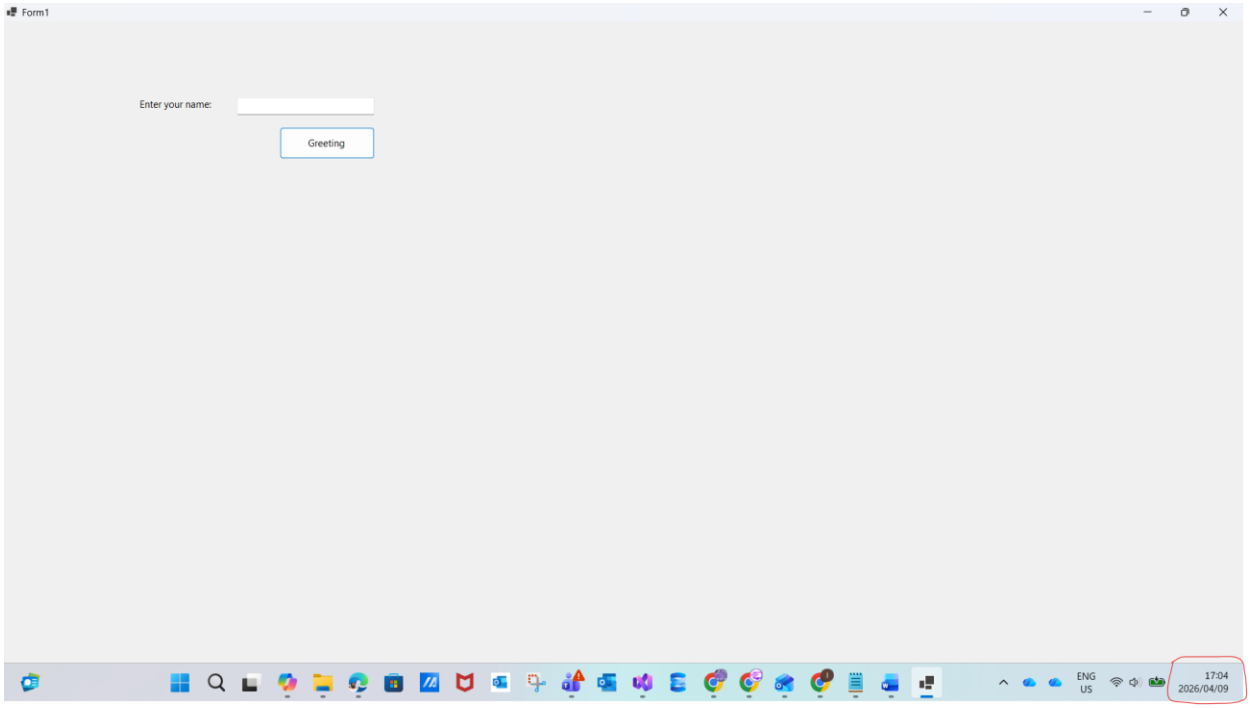
- AK0101 Knowledge of application testing techniques
- AK0102 Knowledge of test report writing

Internal Assessment Criteria

- IAC0101 functional and non-functional tests are identified
- IAC0102 The correct tests are applied
- IAC0103 A test report is produced

Example of Collecting Evidence for your submission pdf:

Question1 :



Ensure each screenshot shows the **full screen**, including the **system date and time**

SECTION A: Knowledge Questions (40 Marks)

Question 1 (30 Marks)

1.1 Define non-functional testing and explain what type of system qualities it evaluates. (2)

Non-functional testing checks/tests how well a system performs rather than what it does. It assesses system qualities such as performance, reliability, usability, security, scalability and maintainability. These are qualities that defines the user's experience as well as the system's behaviour under different kinds of conditions.

1.2 Explain load testing as a non-functional testing type. (1)

Load testing checks a system's behaviour under expected or peak user loads. It verifies that the system can handle a defined number of simultaneous users or transactions without the quality of the systems's performance reducing or degrading with more load/users.

1.3 Describe the difference between load testing and stress testing in non-functional test design. (2)

Load testing applies expected or normal peak load to the system to confirm it performs as it should within acceptable limits.

Stress testing pushes the system beyond its normal capacity to find the system's breaking point and then see how it would recover from failure.

The difference between them is Load testing checks for stability while stress testing identifies a system's limits and failure behaviour.

1.4 Explain the difference between black-box testing and white-box testing. (3)

Black-box testing tests the system from an external/outside perspective only. the tester provides inputs and checks outputs without any knowledge of the what the internal code or logic looks like. It focuses on whether the system behaves according to requirements from a user's perspective. Its not about how the program was made but does it work as it should be used.

White-box testing involves an assessment of the internal code structure, logic paths and implementation. The tester has full visibility of the source code and designs tests to check the logic and implementation of specific code paths, branches, conditions etc.

The difference between them is black-box is behaviour-driven (what does the system do?) and white-box is structure-driven (how does the system do it?).

1.5 Describe equivalence partitioning and explain how it improves functional testing efficiency (3)

Equivalence partitioning is a design test technique which divides input data into groups (also called partitions) in which all the values in a group expect to be processed in the same way as each other. So instead of testing every possible input data, one representative value is selected from each group/partition and a similar result may be expected in a different scenario of the same group.

It improves efficiency by significantly reducing the number of test cases required while still achieving broad coverage for all test/possible inputs. For example, if a field accepts ages 18–65 then the values 0–17(out of range lower end), 18–65(within range) and 66+(out of range upper end) are three partitions. So testing one value per partition is sufficient rather than testing every possible number from say 0-100.

1.6 A Loan Eligibility Processing System accepts customer income values between:

R1 000 and R150 000 Design TWO boundary value test cases to verify correct system behaviour. (4)

Test Case	Input Value	Expected Outcome/output
Test-01	R1 000 (lower boundary)	accepted -> eligible
Test-02	R999 (just below lower boundary)	rejected -> ineligible
Test-03	R150 000 (upper boundary)	accepted -> eligible
Test-04	R150 001 (just above upper boundary)	rejected -> ineligible

1.7 Define error-case testing. (1)

Error-case testing (also known as negative testing) involves intentionally providing invalid, unexpected or out-of-range inputs to confirm that the system does indeed handle errors gracefully by displaying appropriate error messages when invaild input is given instead of crashing or producing incorrect results.

1.8 Explain why testing unexpected user inputs is important when validating software reliability. (2)

Because users will not always follow expected input patterns. So testing unexpected inputs confirms that the system is indeed robust and will not crash, corrupt the data or expose security vulnerabilities when encountering invalid input data. It makes sure the application gives meaningful feedback while maintaining a stable state regardless of what is entered by the user because this is important for real-world functionality and reliability.

1.9 Provide TWO examples of error-case tests suitable for a login system. (2)

- Entering a valid username with an incorrect password: the system should display "Invalid credentials" and deny access instead of crashing or explicitly saying which field is wrong.
- Leaving both the username and the password fields empty then click Login: the system should deny access as well and display a validation message such as "Username and password are required."

1.10 Define a test case in software testing. (2)

A test case is a documented set of conditions, inputs and expected results used to find out if a specific feature or function of a software application works properly as it should. It helps confirm system behaviour in a repeatable and structured way against defined requirements.

1.11 List THREE components typically included in a structured test case. (3)

- Test Case ID or Name
- Input Data and required Preconditions
- Expected Result/outcome

1.12A testing session executed:

Test Cases	Passed	Failed
100	80	20

Prepare a structured test summary section for a system testing report including:

Test results overview

Recommendation to stakeholders (5)

Test Results Overview

Metric	Value
Total Test Cases Executed	100
Test Cases Passed	80
Test Cases Failed	20
Pass Rate	80%
Fail Rate	20%

During this testing session: 100 test cases were executed across the system. 80 cases produced the expected results and passed while 20 cases produced incorrect or unexpected behaviour and are recorded as failures requiring investigation.

Recommendations to Stakeholders

The system achieved an 80% pass rate, indicating that core functionality is largely operational. However, the 20 failed test cases represent a concerning risk to system quality and must be investigated and resolved before the application is approved for production release.

I recommended that:

- The development team conducts a root cause analysis on all 20 failed test cases.
- A re-test (regression test) cycle must be scheduled after the detected defects are fixed.
- The application should not be deployed to production atleast until the pass rate reaches at minimum 95%.
- Keep a defect log to maintain an track resolution progress before reporting back to stakeholders

Total _____ / 30

True / False Questions [10]

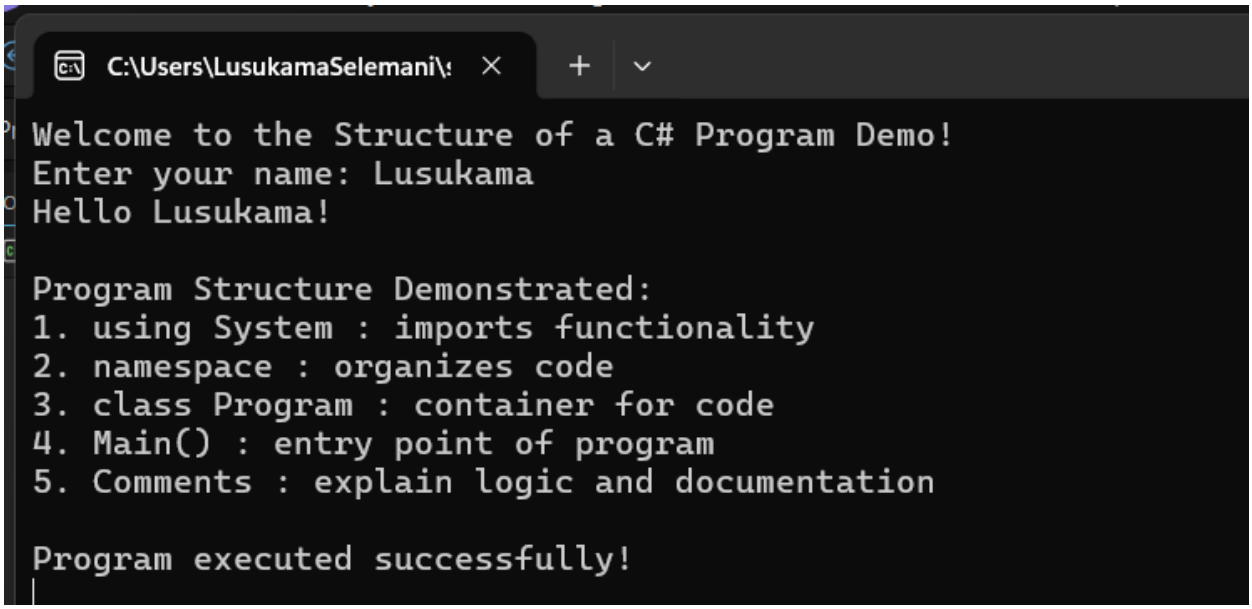
True / False Question	Answer
Exploratory testing allows testers to learn, design, and execute tests at the same time.	TRUE
A good feature specification should be vague to allow flexibility during testing.	FALSE
Specification review meetings help identify missing or incorrect requirements before testing begins.	TRUE
Unit testing is typically performed after system testing.	FALSE
Black-box testing focuses on testing system behavior without examining internal code structure.	TRUE
White-box testing examines internal program logic and code structure.	TRUE
Error-case testing helps ensure the system handles unexpected inputs correctly.	TRUE
UX testing only evaluates system performance and ignores usability.	FALSE
Security testing includes checking authentication and authorization mechanisms.	TRUE
Maintainability testing helps determine how easily a system can be monitored and updated.	TRUE

Total: _____/ 10

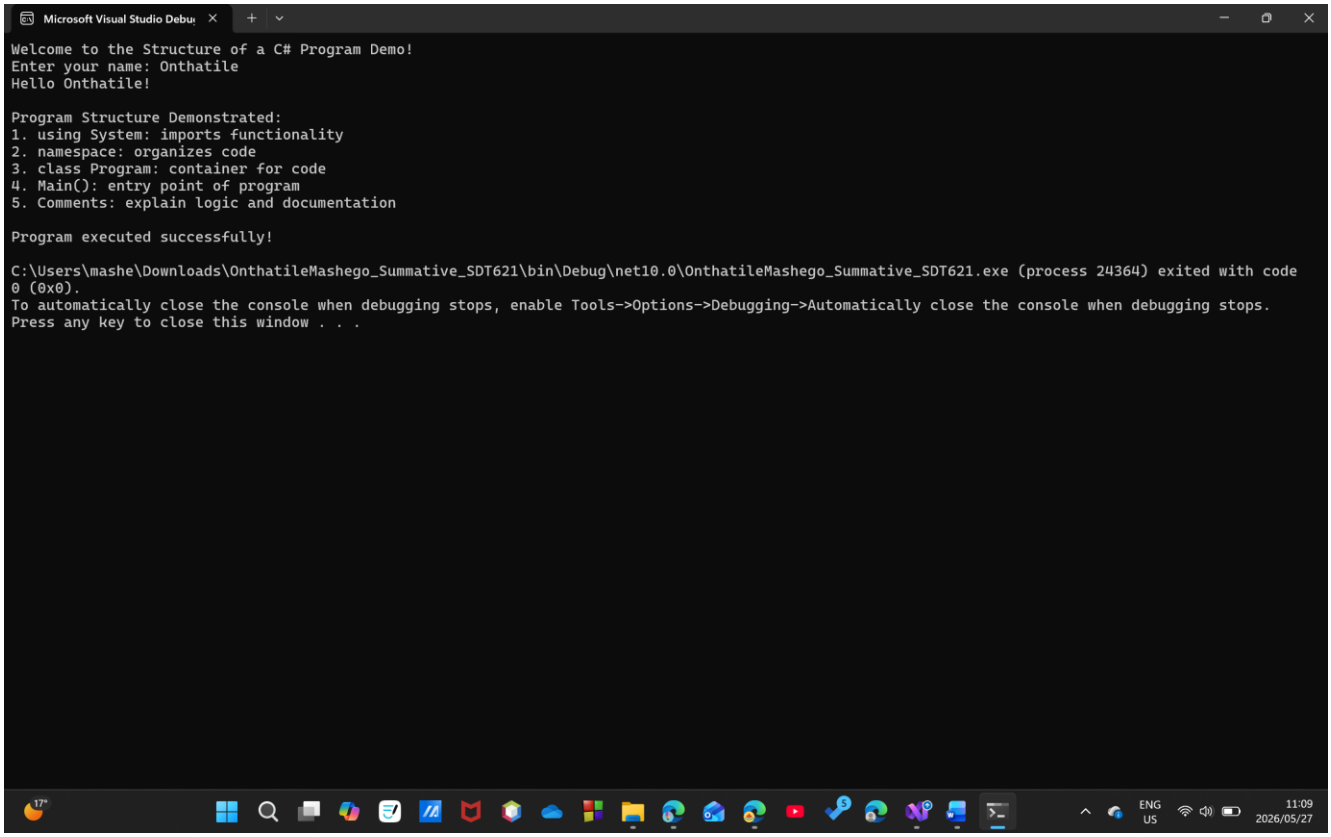
SECTION B: Short Practical Questions (10 Marks)

Question 1

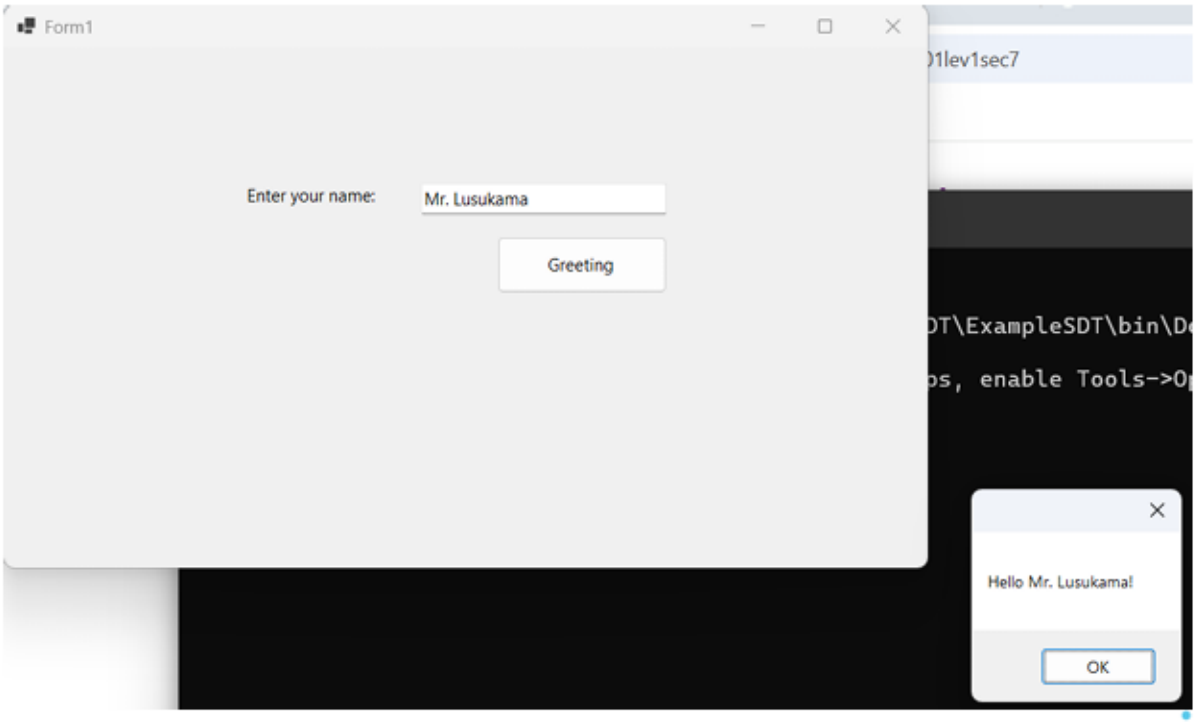
1.1 Create a C# Console Application that the display the below output: (5)



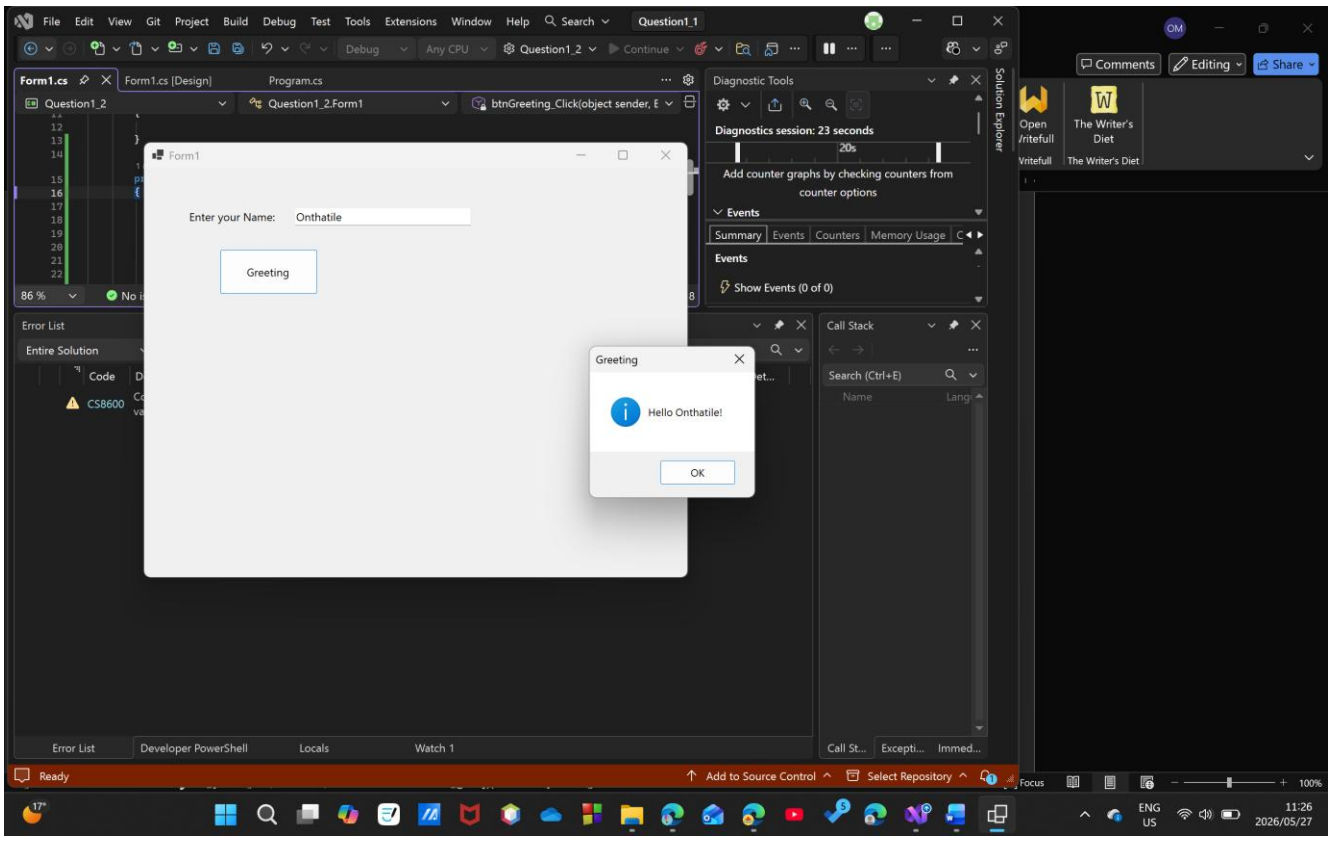
Answer:



1.2 Develop a Windows Forms Application that allows the user to enter their name and display a greeting message. (5)



Answer:



SECTION C: Scenario-Based Practical [50 Marks]

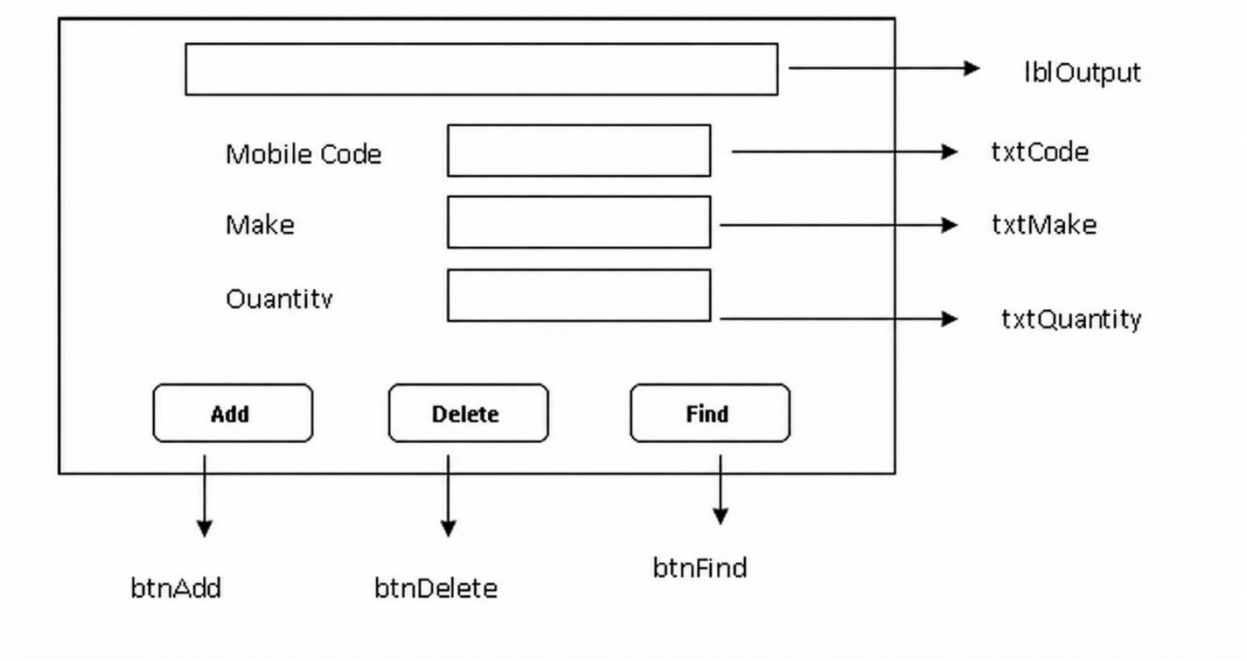
Question 1

You are working as a Junior Software Tester in a retail inventory company. The development team created a Mobile Stock Capture Windows Forms application, but the system contains interface naming issues, missing validations, and incorrect button behavior.

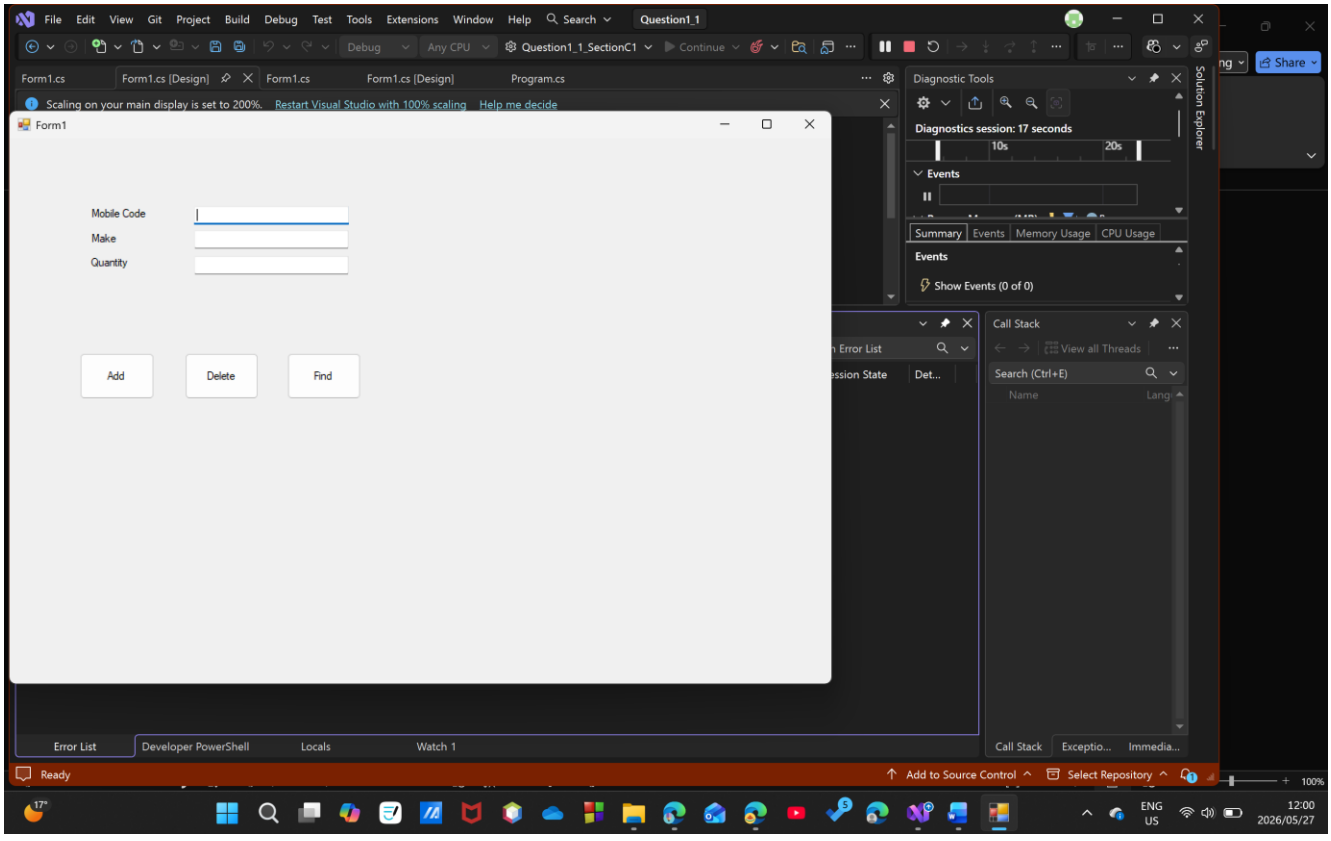
You are required to design, test, debug, and improve the Windows Forms application shown in the diagram.

SECTION C.1 — User Interface Design (10 Marks)

1.1 Create and design the following user interface using appropriate C# Windows Forms App controls and components. (10)



ANSWERS C1:



SECTION C.2 — Button Event Logic Implementation (20 Marks)

1.2 Write the events for each button

1.2.1 When the btnAdd button is clicked, the program should insert a record into the tblMobilePhones with the MobileCode, Make and Quantity from the corresponding data from the form and and display the message “Record Added” in the lblOutput. (8)

1.2.2 When the btnDelete button is clicked, the program should delete the record whose MobileCode is typed in the txtCode text field and display the message “Record Found” in the lblOutput; or “Record NOT Found” if the record is not found. (7)

1.2.3 When the btnFind is clicked, the program should find the record whose

MobileCode is typed in the txtCode text field and display the message “Record Deleted” in the lblOutput. (5)

Note: use functional and non-functional tests for this question

ANSWERS:

WORKING SCREENSHOT PROVIDED IN GITHUB, Ran out of time to screenshot due to database sql errors!

SECTION C.3 : Software Testing Report (15 Marks)

1.3 Produce a Software Testing Report

Using the results from Question 1, compile a professional software testing report for the Mobile Stock Capture Application.

Your report must include the following sections:

1.3. 1 Application Description (3 Marks)

Provide a brief description of:

- Purpose of the application
- Main system functions
- User inputs captured

ANSWER Q1.3.1:

Purpose of the Application: The Mobile Stock Capture Application is a Windows Forms desktop system designed for use by retail inventory staff. It enables the capture, storage, retrieval and deletion of mobile phone stock records in a database.

Main System Functions:

- Add new mobile phone stock records to the database
- Delete existing records by Mobile Code
- Find/retrieve stock records by Mobile Code
- Display feedback messages to the user after each operation

User Inputs Captured:

- Mobile Code (text identifier for each phone)
- Make (brand/manufacturer name)
- Quantity (number of units in stock)

1.3.2 Test Cases Table (5 Marks)

Create a table similar to the example below:

Test Case	Input	Expected Result	Actual Result	Status

Include at least five test cases

ANSWERS Q1.3.2:

Test Case	Input	Expected Result	Actual Result	Status
TC-01: Add valid record	Code: "MB001", Make: "Samsung", Qty: 10	Record inserted; lblOutput = "Record Added"	Record Added displayed	Pass

TC-02: Add with empty fields	All fields blank, click Add	Validation message: "All fields are required"	Validation message shown	Pass
TC-03: Add duplicate code	Code "MB001" already exists, click Add	Error: "MobileCode already exists"	Error message shown	Pass
TC-04: Delete existing record	Code: "MB001", click Delete	Record removed; lblOutput = "Record Found"	Record Found displayed	Pass
TC-05: Delete non-existent record	Code: "MB999", click Delete	lblOutput = "Record NOT Found"	Record NOT Found displayed	Pass
TC-06: Find existing record	Code: "MB001", click Find	Make and Qty populated; lblOutput = "Record Deleted"	Fields populated correctly	Pass
TC-07: Add non-numeric quantity	Code: "MB002", Make: "Apple", Qty: "abc"	Validation error: "Quantity must be a valid positive number"	Error message shown	Pass

1.3.3 Identified Defects (4 Marks)

List any problems found during testing:

1. **Label mismatch on btnFind:** The Find button displays "Record Deleted" in lblOutput, which is misleading. The Delete button should display "Record Deleted" and the Find button should display "Record Found."
2. **No input length validation:** The MobileCode and Make fields accept unlimited characters, which could cause database errors if values exceed column size limits.
3. **No confirmation dialog on Delete:** The application deletes records immediately without asking the user to confirm, creating a risk of accidental data loss.

Quantity field accepts negative numbers: There is no validation preventing a user from entering a negative stock quantity such as -5, which is logically invalid for inventory

1.3.4. Recommendations for Improvement (3 Marks)

1. **Fix button label logic:** Correct the lblOutput messages so that btnDelete displays "Record Deleted" and btnFind displays "Record Found" to match expected functional behaviour.
2. **Add a confirmation dialog:** Before deleting a record, display a MessageBox.Show confirmation prompt (Yes/No) to prevent accidental deletion of valid stock data.
3. **Enforce quantity validation:** Add a check to ensure quantity is greater than zero before allowing a record to be added, and set a maximum character length on all text fields to match database column sizes.

SECTION C.4 — GitHub Version Control Evidence (5 Marks)

1.3.5 Use of GitHub

To obtain the full 5 marks, learners must demonstrate correct use of GitHub version control by completing the following tasks:

- Create and work within a separate branch (not directly on the main branch)
- Make meaningful and professional commit messages that clearly describe the changes made
- Successfully merge the branch into the main (origin/main) branch after completing development
- Ensure the repository contains the complete working solution
- Submit a valid and accessible GitHub repository link

Link To My Github Repo:

https://github.com/Onthatile-CTU/OnthatileMashego_SDT621_SummativeExam/

Onthatile-CTU/OnthatileMashego_SDT621_SummativeExam

End Exam

SECTION A — Knowledge Questions (40 Marks)

Purpose of Section A

Assess theoretical understanding of testing techniques, validation strategies, and structured test documentation required before practical debugging begins.

QCTO Mapping

Question Area	Marks	QCTO Criterion	Evidence Demonstrated
Non-functional testing definition	2	AK0101	Knowledge of testing concepts
Load testing explanation	1	AK0101	Understanding performance testing
Load vs Stress testing comparison	2	AK0101	Distinguishing test strategies
Black-box vs White-box testing	3	AK0101	Test method classification
Equivalence partitioning	3	AK0101	Functional test design
Boundary value test cases	4	AK0101	Boundary testing application
Error-case testing definition	1	AK0101	Exception scenario awareness
Unexpected input validation importance	2	AK0101	Reliability assurance concepts
Login error-case examples	2	AK0101	Functional validation thinking
Definition of test case	2	AK0102	Test documentation knowledge
Components of test case	3	AK0102	Structured testing documentation
Test summary & stakeholder recommendation	5	AK0102	Test reporting interpretation

Section A Mapping Summary

Criterion	Marks
AK0101 Knowledge of testing techniques	25
AK0102 Knowledge of test report writing	15

Total Section A = 40 Marks

SECTION B — Short Practical Questions (10 Marks)

Purpose of Section B

Assess ability to implement and validate basic working software functionality before applying structured testing.

QCTO Mapping Table

Task	Marks	QCTO Criterion	Evidence Demonstrated
Console output application	5	IAC0102	Correct functional output validation
Windows Forms greeting application	5	IAC0102	UI interaction testing

Section B Mapping Summary

Criterion	Marks
IAC0102 Correct tests applied	10

Total Section B = 10 Marks

SECTION C — Scenario-Based Practical (50 Marks)

Purpose of Section C

Assess full PM-04 competency:

Test or debug source code to ensure client needs are met

Learners must design, debug, validate, test, and document results.

SECTION C.1 — User Interface Design (10 Marks)

Task	Marks	QCTO Criterion	Evidence Demonstrated
Correct UI controls selected	2	IAC0101	Functional test readiness
Correct control naming	2	IAC0101	Interface clarity
Layout alignment with specification	3	IAC0101	UI correctness verification
Proper component usage	3	IAC0101	Interface usability validation

Mapping Summary

Criterion	Marks
IAC0101 Functional & non-functional tests identified	10

SECTION C.2 — Button Event Logic Implementation (20 Marks)

Task	Marks	QCTO Criterion	Evidence Demonstrated
Add button functionality	8	IAC0102	Record insertion testing
Delete button functionality	7	IAC0102	Conditional logic validation
Find button functionality	5	IAC0102	Search behaviour verification

Mapping Summary

Criterion	Marks
IAC0102 Correct tests applied	20

SECTION C.3 — Software Testing Report (15 Marks)

Task	Marks	QCTO Criterion	Evidence Demonstrated
Application description	3	AK0102	Documentation understanding
Test cases table	5	IAC0102	Test execution structure
Identified defects	4	IAC0101	Fault detection ability

Recommendations	3	IAC0103	Improvement reporting

SECTION C.4 — GitHub Version Control Evidence (5 Marks)

Task	Marks	QCTO Criterion	Evidence Demonstrated
Branch created and used	1	IAC0102	Version control workflow
Professional commit messages	1	IAC0102	Change tracking clarity
Multiple commits recorded	1	IAC0102	Development progression evidence
Branch merged into main	1	IAC0102	Integration workflow
Repository complete & accessible	1	IAC0103	Submission verification

SECTION C Mapping Summary

Criterion	Marks
IAC0101 Tests identified	14
IAC0102 Tests applied	26
IAC0103 Test report produced	10

Total Section C = 50 Marks

FINAL OVERALL PM-04 ASSESSMENT SUMMARY

QCTO Criterion	Description	Marks
AK0101	Knowledge of testing techniques	25
AK0102	Knowledge of test reporting	18
IAC0101	Functional & non-functional tests identified	14
IAC0102	Correct tests applied	36
IAC0103	Test report produced	7

TOTAL = 100 MARKS